

Resumen del Proyecto: Gestión de Tienda de Informática

<https://github.com/Marmotrolo/ProyectoUD3>

El tema de mi proyecto es la gestión integral de una tienda de informática. He decidido organizar la información en torno al **Producto**, que es el eje central de la base de datos, y lo he conectado con el resto de entidades mediante diferentes relaciones

Las relaciones que he decidido usar:

Producto - Garantía (1:1): He decidido que cada producto tenga su propia garantía personalizada. Al ser una relación uno a uno, si borro el producto, la garantía se elimina automáticamente

Fabricante - Categoría - Producto (1:N): He agrupado los productos por marcas (Fabricantes) y por tipos (Categorías). Un fabricante puede tener un catálogo de muchos productos, pero cada producto pertenece solo a una marca y a una sección específica.

Cliente - Producto (N:M): Esta es la relación de ventas. He permitido que un cliente pueda comprar varios productos y que un producto pueda ser adquirido por diferentes personas. Para esto, Hibernate me crea una tabla intermedia que registra todas estas compras.

Empleados: He incluido una entidad de empleados para gestionar la plantilla de la tienda (vendedores, técnicos, etc.) de forma independiente.

Aquí tienes el detalle de los campos y tipos de datos que he definido para cada tabla en MySQL a través de Hibernate:

Campos por Entidad

Tabla Productos: Contiene el id (Long, clave primaria), el nombre (String), el precio (Double) y las claves ajenas categoria_id y fabricante_id para conectar con sus respectivas tablas.

Tabla Garantías: Guarda el id (Long), la fechaVencimiento (LocalDate), las condiciones (String) y el producto_id. Es una relación 1:1, por lo que el ID del producto es único aquí para que no se repita.

Tabla Categorías: Es muy sencilla, solo tiene el id (Long) y el nombre (String) de la sección, como "Periféricos" o "Componentes".

Tabla Fabricantes: Registra el id (Long), el nombre (String) de la marca y el país (String) de origen del fabricante.

Tabla Clientes: Almacena el id (Long), el nombre (String) y el email (String). No tiene claves ajenas directas porque los datos de las compras se guardan en la tabla intermedia.

Tabla Empleados: Contiene el id (Long), el nombre (String), el puesto (String) y el salario (Double).

Consultas con HQL

Producto más caro: **FROM Producto p ORDER BY p.precio DESC**

Listado de nombres: **SELECT p.nombre FROM Producto p**

Nombre y Precio: **SELECT p.nombre, p.precio FROM Producto p**

Búsqueda por precio máximo: **WHERE p.precio <= :precioMax**

Búsqueda por nombre (Buscador): **LIKE LOWER(:busqueda)**

Estadísticas (AVG y COUNT): **SELECT AVG(p.precio) y SELECT COUNT(p)**

Filtros con ordenación: **BETWEEN :precioMin AND :precioMax**

Consultas con Criteria API

Actualización de precio individual: **cb.createCriteriaUpdate**

Aplicación de descuentos masivos: **cb.prod(root.get("precio"), 1 - porcentaje)**

Limpieza de stock (Borrado): **cb.createCriteriaDelete**

Búsqueda avanzada por rango: **cb.greaterThanOrEqualTo y cb.lessThanOrEqualTo**

